BÀI TẬP 2 · BUỔI 12 · TRACK SE

Hệ thống tiếp nhận và phân loại yêu cầu bảo hành

Mekong Mobile · Luồng nghiệp vụ L5 – Kho linh kiện thay thế

Họ và tên	Trần Trí Dũng
MSSV	2374802010070
Lớp	71ITGR40203_03
Track / Luồng	SE / L5 – Kho linh kiện thay thế
Phiên bản nộp	**v0.2 · Bài tập 2**

Repository:
https://github.com/TriDung1208/smartcrm-2374802010070-kho-linh-kien-thay-the
Tag dự kiến: v0.2

MỤC 1

Tóm tắt hệ thống

Phân hệ L5 hỗ trợ nhân viên kho quản lý linh kiện thay thế tại trung tâm dịch vụ: tra cứu số lượng và ngưỡng cảnh báo, nhập kho, xuất kho theo yêu cầu bảo hành, và xem lịch sử giao dịch.

> **Mục tiêu:** bảo đảm số lượng tồn được cập nhật nhất quán, không cho xuất vượt tồn và cung cấp phản hồi rõ ràng khi dữ liệu đầu vào không hợp lệ.

Phạm vi đã hiện thực

- Ba màn hình gọi API thật: tồn kho; xuất linh kiện; nhập kho và lịch sử.
- CRUD hai thực thể danh mục: trung tâm dịch vụ và nhóm lỗi.
- Kiểm tra dữ liệu, tồn kho trước khi xuất; ghi biến động tồn và giao dịch trong cùng transaction.
- Migration và seed có thể chạy từ CLI; chạy seed lặp không nhân đôi dữ liệu.
- Giao diện có dữ liệu, trạng thái rỗng và thông báo lỗi khi backend ngừng hoạt động.

Giới hạn được công khai

Mã phiếu bảo hành hiện chỉ được lưu làm mã tham chiếu; chưa kết nối hoặc xác minh với phân hệ bảo hành. Bản hiện tại chưa phân quyền theo vai trò và lịch sử chưa có bộ lọc thời gian.

Đối tượng sử dụng

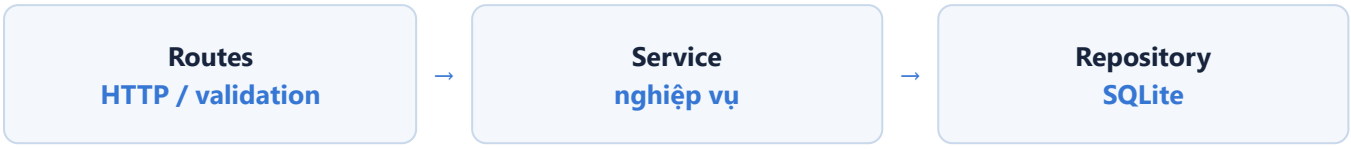
Nhân viên kho
Tra cứu tồn, nhập và xuất linh kiện, xem lịch sử.

Quản lý trung tâm
Theo dõi tồn và các mặt hàng cần bổ sung.

MỤC 2

Kiến trúc đã hiện thực

Backend tổ chức theo ba lớp, tách tiếp nhận HTTP khỏi quy tắc nghiệp vụ và thao tác lưu trữ. SQLite là cơ sở dữ liệu của bản demo; migration tạo schema và seed nạp dữ liệu mẫu.



Frontend
Ứng dụng web tĩnh gọi REST API; server giao diện độc lập ở cổng 5173 proxy các route API tới backend cổng 3000.

Backend
Node.js, API dưới `/api/v1`, xử lý lỗi theo mã HTTP và mã lỗi có cấu trúc.

Dữ liệu
SQLite lưu danh mục, tồn và giao dịch; nhập/xuất cập nhật tồn và lịch sử trong transaction.

Thiết lập
`npm ci` dùng package-lock; `npm run migrate` tạo schema; `npm run seed` nạp danh mục mẫu.

Luồng xuất linh kiện

1. Route nhận yêu cầu và kiểm tra dữ liệu đầu vào.
2. Service kiểm tra số lượng hợp lệ và tồn khả dụng.
3. Repository ghi giao dịch xuất, trừ tồn trong một transaction.
4. API trả tồn mới; nếu thiếu hàng trả `409 INSUFFICIENT_STOCK`.

API chính

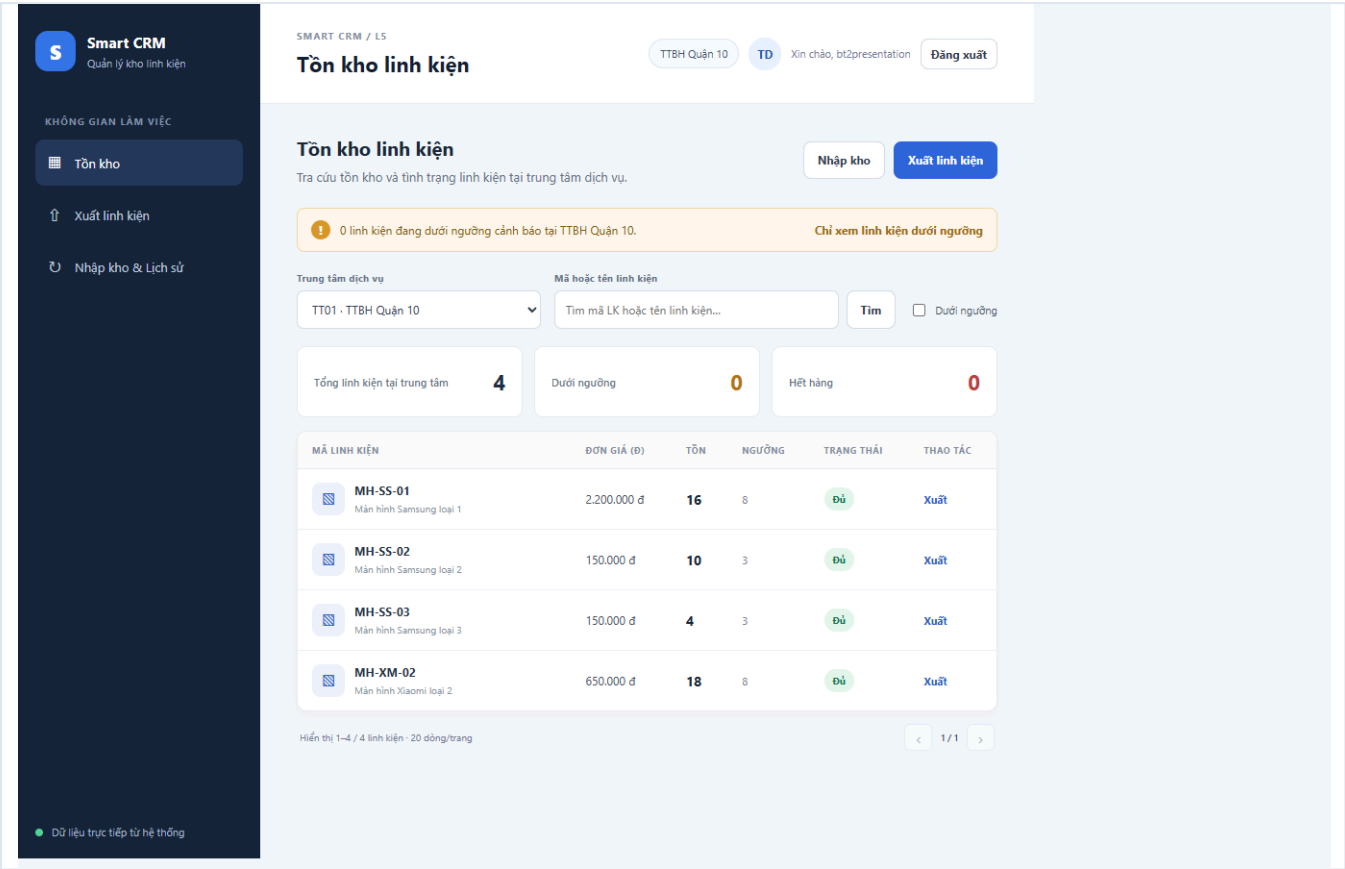
Nhóm	Endpoint tiêu biểu
Danh mục	`/service-centers`, `/issue-categories` — GET/POST/PUT/DELETE và chi tiết theo ID
Kho	`/stocks`, `/stock-receipts`, `/stock-issues`, `/stock-alerts`, `/stock-transactions`
Sức khỏe	`/health`

MỤC 3

Module cốt lõi

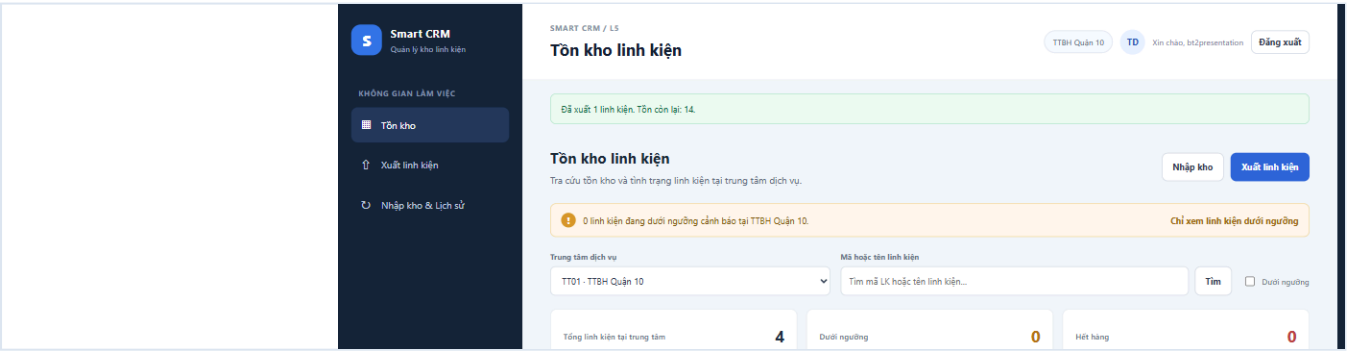
Tra cứu tồn theo trung tâm, kiểm tra trạng thái từng linh kiện và thực hiện xuất kho. Các API danh mục hỗ trợ CRUD đầy đủ cho trung tâm dịch vụ và nhóm lỗi.

Ảnh chạy thật – tồn kho



Hình 1. Màn hình tồn kho hiển thị danh sách, số lượng, ngưỡng và trạng thái từ API.

Ảnh chạy thật – xuất kho thành công



Hình 2. Thông báo thao tác thành công và tồn mới sau khi xuất một linh kiện.

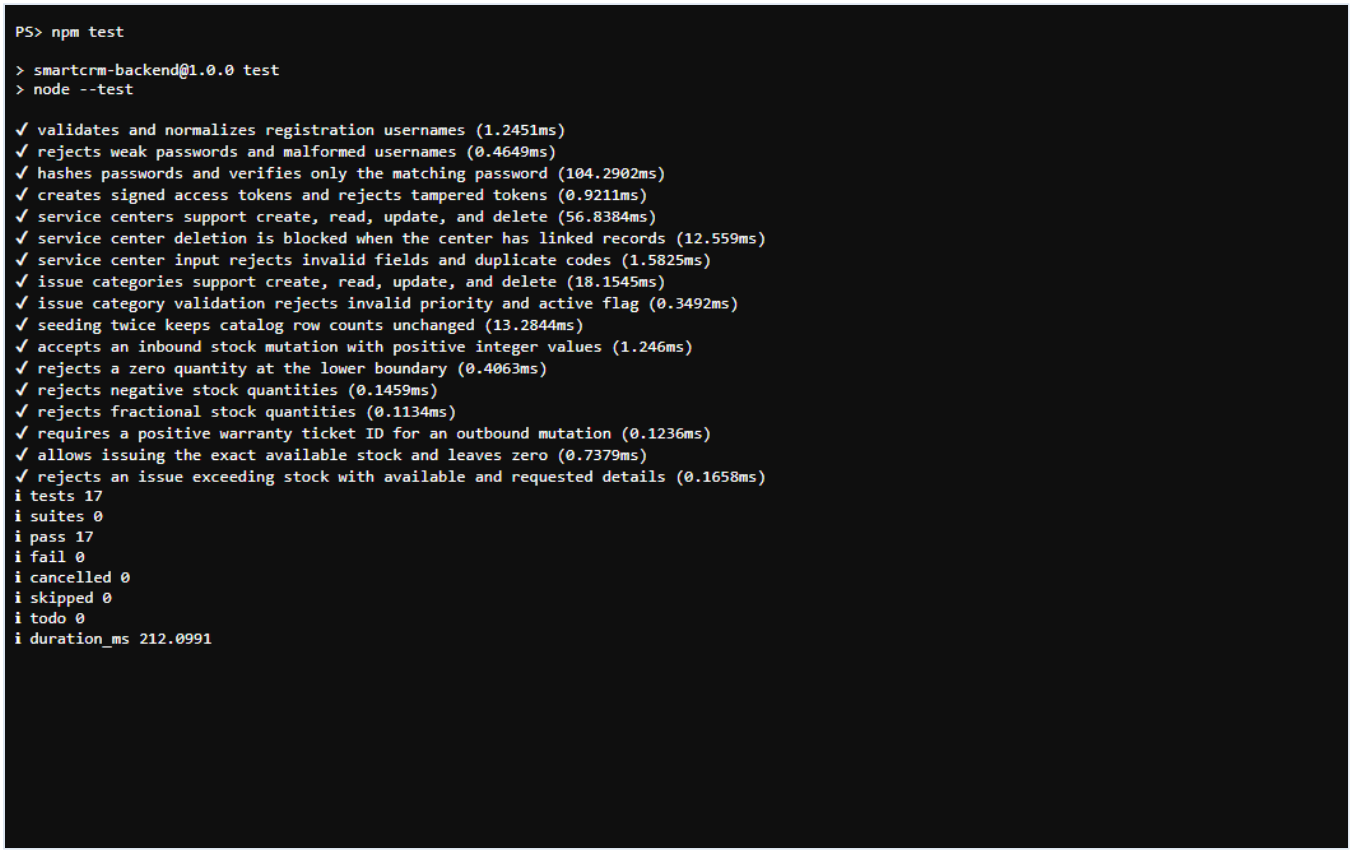
> **Ràng buộc:** không chấp nhận số lượng bằng 0, âm hoặc phân số; không cho xuất quá số lượng khả dụng. Trung tâm đang được tham chiếu không thể xóa (HTTP 409).

MỤC 4

Bộ kiểm thử

Chạy một lệnh `npm test` từ thư mục `src/backend`. Dự án dùng Node.js test runner và database tạm cho kiểm thử CRUD/seed.

Loại	Ví dụ kiểm thử	Kết quả
Luồng đúng	CRUD trung tâm/nhóm lỗi; nhập kho hợp lệ; xuất đúng lượng còn tồn.	PASS
Biên	Xuất đúng lượng tồn để số dư về 0; username/mật khẩu theo giới hạn.	PASS
Dữ liệu sai	Quantity bằng 0, âm, phân số; mã phiếu không hợp lệ; danh mục sai.	PASS
Tính lặp	Chạy seed hai lần; số dòng danh mục giữ nguyên.	PASS



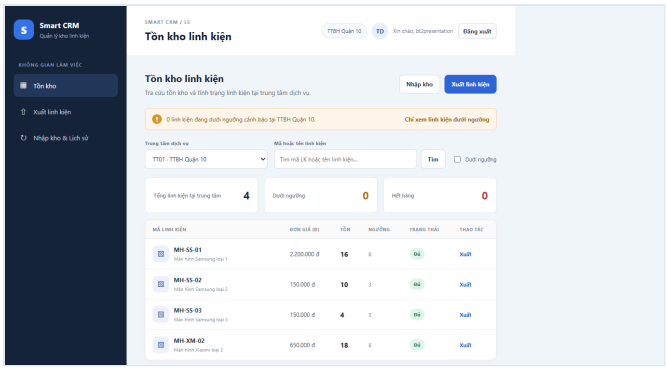
Hình 3. Ảnh chụp lệnh chạy test: 17 kiểm thử, 17 PASS, 0 FAIL.

> **Kết quả:** 17 tests · 17 pass · 0 fail. Bao gồm unit validation, CRUD, tình huống dữ liệu biên/sai và seed idempotent.

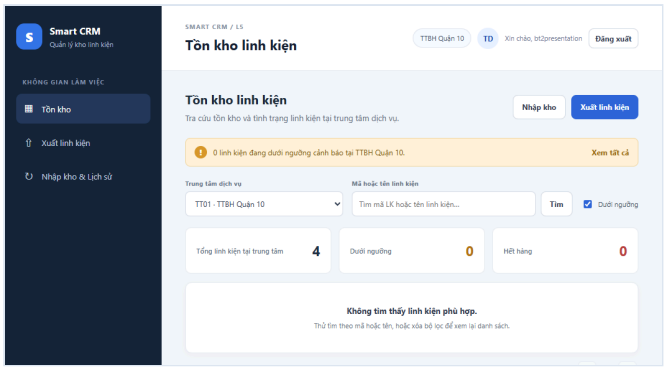
MỤC 5

Trình bày kết quả

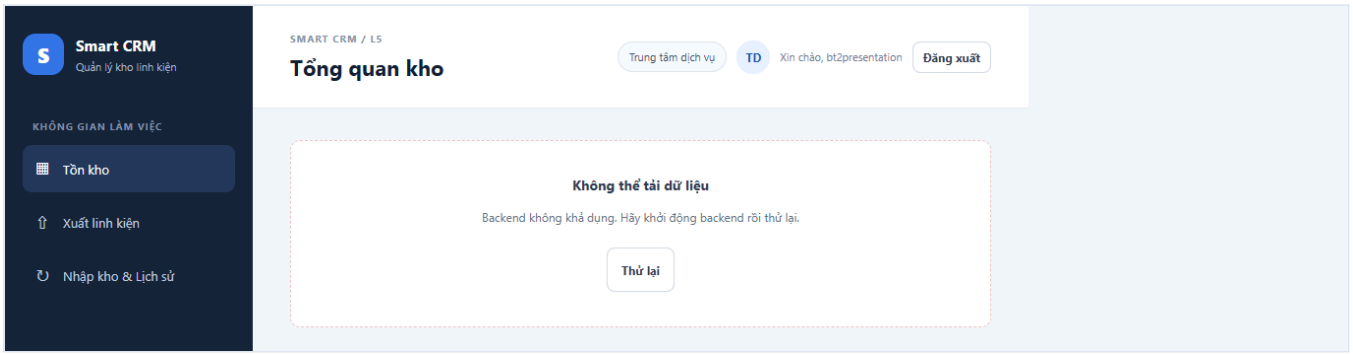
Giao diện có trạng thái có dữ liệu, rỗng và lỗi backend; trang vẫn tải khi backend tắt và cho biết cách khôi phục.



1. Có dữ liệu: tồn kho, trạng thái và ngưỡng cảnh báo.



2. Trống: bộ lọc không khớp; hướng dẫn xóa bộ lọc để xem lại danh sách.



3. Lỗi: frontend vẫn tải độc lập, hiển thị backend không khả dụng và nút thử lại.

Xử lý dữ liệu nhập sai

Yêu cầu nhập kho với `quantity: 0` trả HTTP 400 và mã `INVALID_QUANTITY`. JSON sai định dạng trả HTTP 400 `INVALID_JSON`; API không sập.

MỤC 6

Cách dựng lại hệ thống

Yêu cầu: Node.js 18.x hoặc 20.x và npm 9 trở lên. Làm theo README trong bản clone mới.

```
git clone https://github.com/TriDung1208/smartcrm-2374802010070-kho-linh-kien-thay-the.git thu-nghiem
Set-Location thu-nghiem
Copy-Item .env.example .env
Set-Location src\backend
npm ci
npm run migrate
npm run seed
npm test
npm start
```

Backend và giao diện tích hợp: `http://localhost:3000`. Health check: `http://localhost:3000/health`.

Chạy frontend độc lập

Mở terminal thứ hai tại `src/backend`, chạy `npm run frontend`, sau đó mở `http://localhost:5173`. Khi backend ngừng, trang vẫn mở và báo lỗi rõ.

Cấu hình và dữ liệu mẫu

- `.env.example` liệt kê tên biến cấu hình; sao chép thành `.env` trước khi chạy.
- `src/backend/package-lock.json` khóa dependency để cài bằng `npm ci`.
- SQLite được tạo bởi migration; seed nạp danh mục mẫu và có thể chạy lại không nhân đôi dòng.
- Không đưa `.env` hoặc database runtime vào commit.

Đường dẫn kiểm chứng trong repo

`README.md`, `docs/api-contract.md`, `docs/srs.md`, `docs/bug-log.md`, `docs/ai-disclosure.md`.

MỤC 7

Thay đổi so với BT1

Các điểm dưới đây được ghi chi tiết tại `docs/thay-doi-thiet-ke.md`; ưu tiên tính nhất quán tài liệu và khả năng dựng/chạy sản phẩm.

Chủ đề	Thay đổi / quyết định	Kết quả
Use Case, actor, ERD	Đồng bộ mã Use Case và actor; đối chiếu số bảng trên ERD với DDL.	Tài liệu mô hình thống nhất hơn.
Yêu cầu và wireframe	Làm rõ dữ liệu hiển thị, đánh đổi NFR; rút gọn user story theo phạm vi.	Tránh mô tả dữ liệu không có trong mô hình.
Clone mới	Bổ sung runtime, lockfile, `.env.example`, migration/seed CLI và lệnh test trong README.	Quy trình dựng lại có thể lặp lại.
CRUD và dữ liệu	Hoàn thiện CRUD hai thực thể; validation và lỗi xung đột; seed chạy lặp an toàn.	Hỗ trợ kiểm chứng bằng API/test.
Frontend độc lập	Tách máy chủ tĩnh và xử lý trạng thái khi backend tắt.	Trang vẫn tải và báo lỗi rõ ràng.
Giới hạn tính năng	Ghi rõ ticket chỉ là tham chiếu; chưa có xác minh phiếu và lọc lịch sử theo thời gian.	Tài liệu phản ánh đúng implementation.
Minh chứng	Bổ sung nhật ký lỗi, khai báo AI và ảnh chụp UI/test.	Có cơ sở trình bày và truy vết.

Lỗi đã xử lý – BUG-BT2-01

Lệnh migrate/seed trước đây chỉ export hàm nên gọi từ npm không tạo schema/dữ liệu. Đã bổ sung entrypoint CLI, báo lỗi với exit code; kiểm chứng bằng migrate, seed hai lần (6 trung tâm, 7 dòng tồn) và test hồi quy.

Giới hạn và bước tiếp theo

Bản này tập trung L5; xác minh ticket và phân quyền theo vai trò là phần mở rộng cần tích hợp với các phân hệ khác.

PHỤ LỤC BẮT BUỘC

Khai báo sử dụng AI

GitHub Copilot / AI assistant trong VS Code được dùng để hỗ trợ rà soát yêu cầu, triển khai một số thay đổi mã nguồn và tài liệu, và bổ sung kiểm thử. Người thực hiện chịu trách nhiệm đọc lại, chạy kiểm thử và xác nhận nội dung trước khi nộp.

Công cụ	Việc được hỗ trợ	Tệp / hàm	Cách kiểm chứng
Copilot / AI assistant	Đối chiếu checklist track SE; hoàn thiện tài liệu dựng/chạy.	`README.md`, `docs/srs.md`, `docs/api-contract.md`	So sánh lệnh, route và giới hạn với mã nguồn.
Copilot / AI assistant	CRUD, validation, xử lý xung đột danh mục.	`serviceCenterService.js`, `issueCategoryService.js` và repository/routes tương ứng	`npm test`; kiểm tra CRUD, dữ liệu sai và bản ghi đang tham chiếu.
Copilot / AI assistant	Migration/seed CLI và tính lặp an toàn.	`database/seed.js`, `database/migrations/initDatabase.js`	Chạy migrate/seed lặp và test seed idempotent.
Copilot / AI assistant	Frontend độc lập, thông báo khi backend không khả dụng.	`src/frontend/server.js`, `src/frontend/api/client.js`	Mở giao diện khi backend tắt; xác nhận trang tải và lỗi được trình bày.
Copilot / AI assistant	Soạn nhật ký lỗi, thiết kế và khai báo AI.	`docs/bug-log.md`, `docs/thay-doi-thiet-ke.md`, `docs/ai-disclosure.md`	Đối chiếu nội dung với thay đổi và kết quả kiểm thử.

> **Cam kết:** Tôi đã rà soát nội dung và chịu trách nhiệm về sản phẩm, minh chứng cũng như việc giải thích các phần mã nguồn khi được yêu cầu.

Bảng khai báo chi tiết cũng có tại `docs/ai-disclosure.md`.